

Отчёт по лабораторной работе 8

дисциплина: Архитектура компьютера

Нассер Мохамад

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
2.1	Самостоятельное задание	20
3	Выводы	25

Список иллюстраций

2.1	Программа в файле lab9-1.asm	7
2.2	Запуск программы lab9-1.asm	7
2.3	Программа в файле lab9-1.asm	8
2.4	Запуск программы lab9-1.asm	9
2.5	Программа в файле lab9-2.asm	10
2.6	Запуск программы lab9-2.asm в отладчике	11
2.7	Дизассемблированный код	12
2.8	Дизассемблированный код в режиме интел	13
2.9	Точка останова	14
2.10	Изменение регистров	15
2.11	Изменение регистров	16
2.12	Изменение значения переменной	17
2.13	Вывод значения регистра	18
2.14	Вывод значения регистра	19
2.15	Вывод значения регистра	20
2.16	Программа в файле prog-1.asm	21
2.17	Запуск программы prog-1.asm	21
2.18	Код с ошибкой	22
2.19	Отладка	23
2.20	Код исправлен	24
2.21	Проверка работы	24

Список таблиц

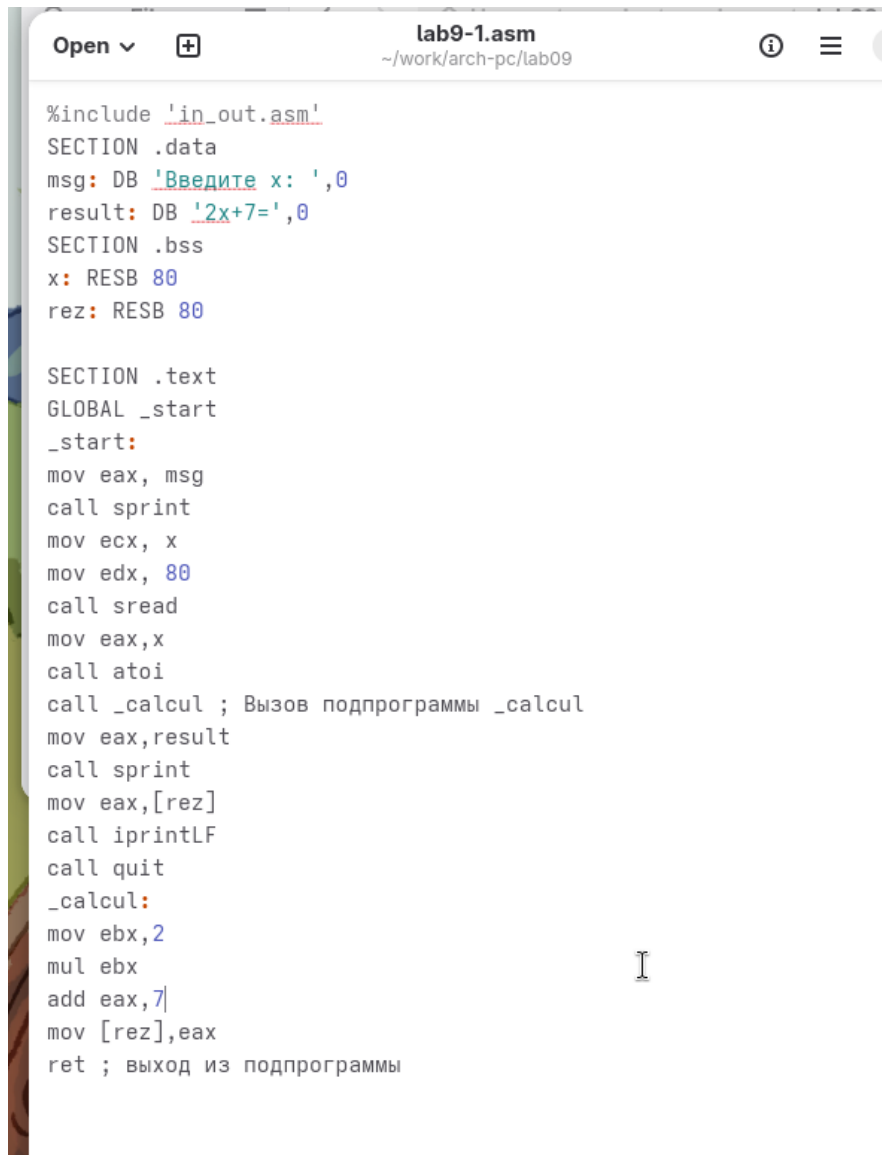
1 Цель работы

Целью работы является приобретение навыков написания программ с использованием подпрограмм. Знакомство с методами отладки при помощи GDB и его основными возможностями.

2 Выполнение лабораторной работы

Я создал каталог для выполнения лабораторной работы № 9 и перешел в него. Затем я создал файл lab9-1.asm.

В качестве примера рассмотрим программу вычисления арифметического выражения $f(x) = 2x + 7$ с помощью подпрограммы calcul. В данном примере x вводится с клавиатуры, а само выражение вычисляется в подпрограмме.(рис. 2.1) (рис. 2.2)

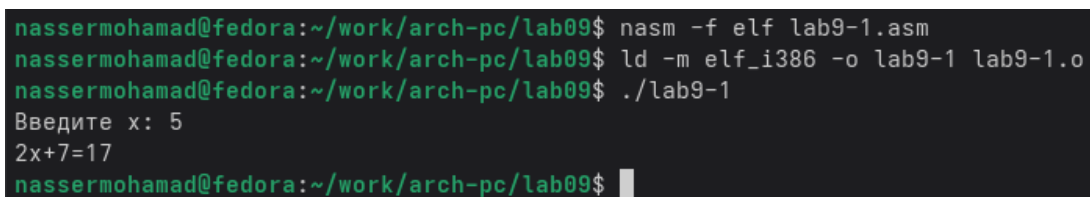


```
lab9-1.asm
~/work/arch-pc/lab09

#include 'in_out.asm'
SECTION .data
msg: DB 'Введите x: ',0
result: DB '2x+7=',0
SECTION .bss
x: RESB 80
rez: RESB 80

SECTION .text
GLOBAL _start
_start:
mov eax, msg
call sprint
mov ecx, x
mov edx, 80
call sread
mov eax, x
call atoi
call _calcul ; Вызов подпрограммы _calcul
mov eax, result
call sprint
mov eax, [rez]
call iprintLF
call quit
_calcul:
mov ebx, 2
mul ebx
add eax, 7
mov [rez], eax
ret ; выход из подпрограммы
```

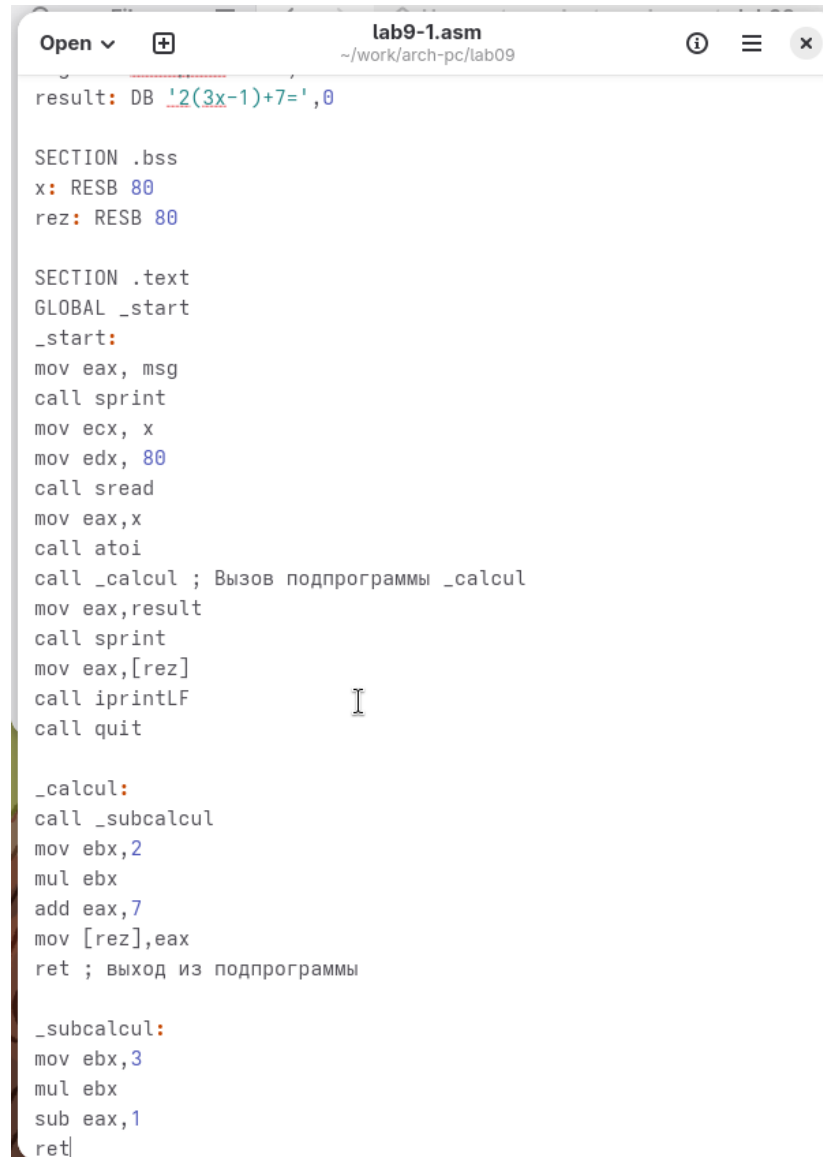
Рисунок 2.1: Программа в файле lab9-1.asm



```
nassermohamad@fedora:~/work/arch-pc/lab09$ nasm -f elf lab9-1.asm
nassermohamad@fedora:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab9-1 lab9-1.o
nassermohamad@fedora:~/work/arch-pc/lab09$ ./lab9-1
Введите x: 5
2x+7=17
nassermohamad@fedora:~/work/arch-pc/lab09$
```

Рисунок 2.2: Запуск программы lab9-1.asm

Изменил текст программы, добавив подпрограмму `subcalcul` в подпрограмму `calcul`, для вычисления выражения $f(g(x))$, где x вводится с клавиатуры, $f(x) = 2x + 7$, $g(x) = 3x - 1$. (рис. 2.3) (рис. 2.4)



```
Open ▾ + lab9-1.asm
~/work/arch-pc/lab09

result: DB '2(3x-1)+7=',0

SECTION .bss
x: RESB 80
rez: RESB 80

SECTION .text
GLOBAL _start
_start:
mov eax, msg
call sprint
mov ecx, x
mov edx, 80
call sread
mov eax, x
call atoi
call _calcul ; Вызов подпрограммы _calcul
mov eax, result
call sprint
mov eax, [rez]
call iprintLF
call quit

_calcul:
call _subcalcul
mov ebx, 2
mul ebx
add eax, 7
mov [rez], eax
ret ; выход из подпрограммы

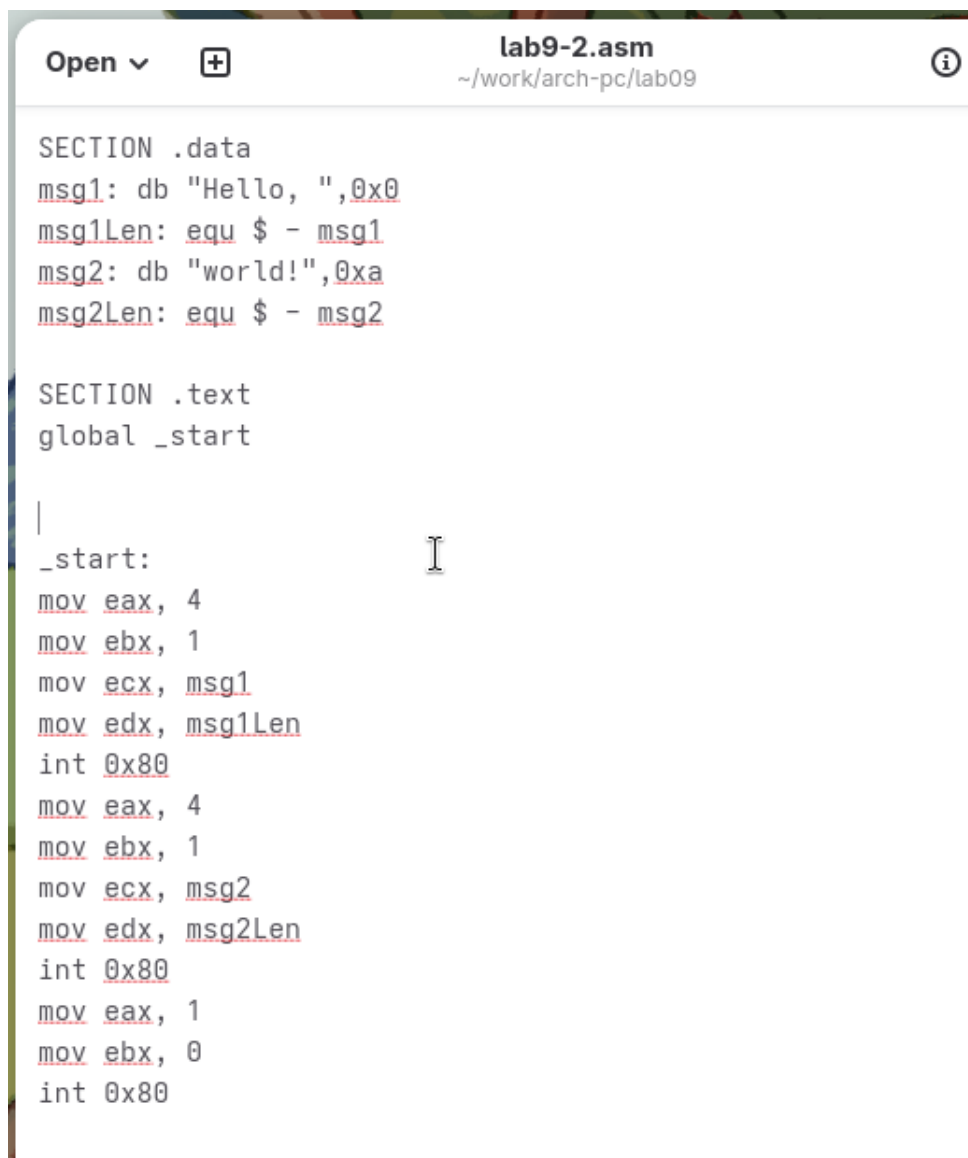
_subcalcul:
mov ebx, 3
mul ebx
sub eax, 1
ret
```

Рисунок 2.3: Программа в файле lab9-1.asm


```
nassermohamad@fedora:~/work/arch-pc/lab09$  
nassermohamad@fedora:~/work/arch-pc/lab09$ nasm -f elf lab9-1.asm  
nassermohamad@fedora:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab9-1 lab9-1.o  
nassermohamad@fedora:~/work/arch-pc/lab09$ ./lab9-1  
Введите x: 5  
2(3x-1)+7=35  
nassermohamad@fedora:~/work/arch-pc/lab09$
```

Рисунок 2.4: Запуск программы lab9-1.asm

Создал файл lab9-2.asm с текстом программы из Листинга 9.2. (Программа печати сообщения Hello world!). (рис. 2.5)



```
Open ▾ + lab9-2.asm
~/work/arch-pc/lab09

SECTION .data
msg1: db "Hello, ",0x0
msg1Len: equ $ - msg1
msg2: db "world!",0xa
msg2Len: equ $ - msg2

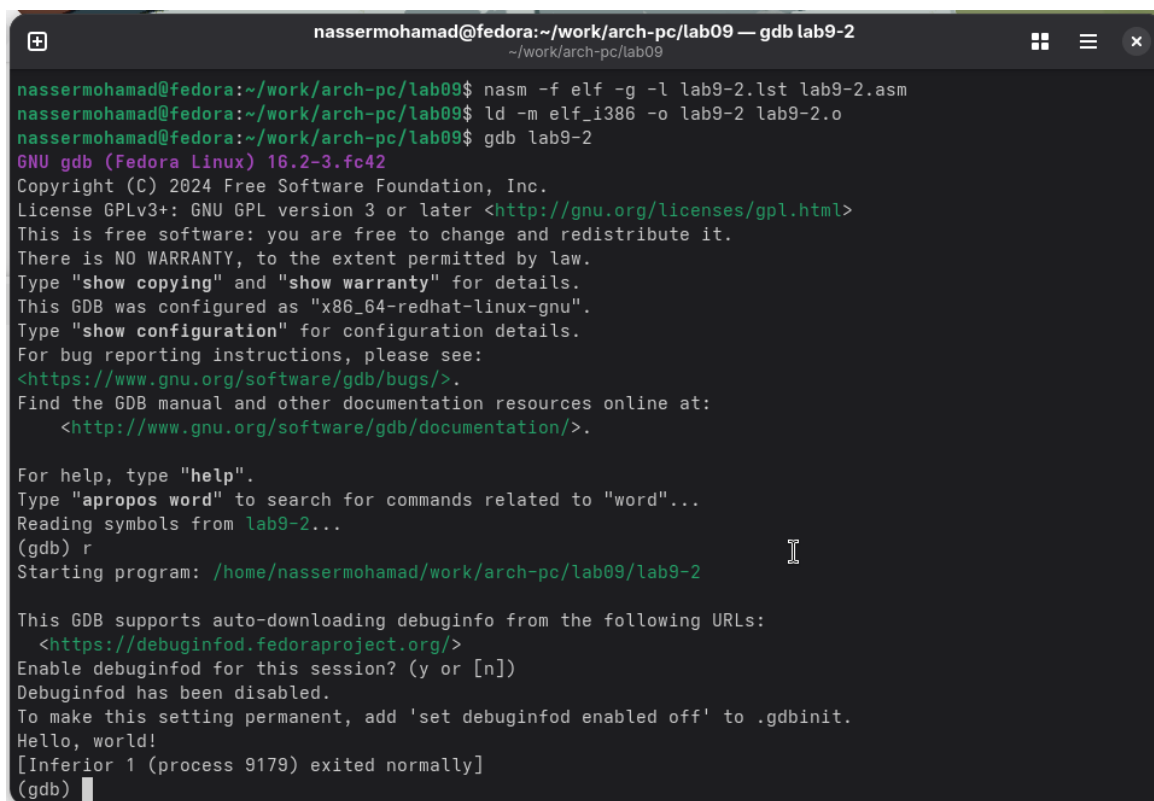
SECTION .text
global _start

|
_start:
mov eax, 4
mov ebx, 1
mov ecx, msg1
mov edx, msg1Len
int 0x80
mov eax, 4
mov ebx, 1
mov ecx, msg2
mov edx, msg2Len
int 0x80
mov eax, 1
mov ebx, 0
int 0x80
```

Рисунок 2.5: Программа в файле lab9-2.asm

Получил исполняемый файл и добавил отладочную информацию с помощью ключа „-g“ для работы с GDB.

Загрузил исполняемый файл в отладчик GDB и проверил работу программы, запустив ее с помощью команды „run“ (сокращенно „r“). (рис. 2.6)



```
nassermohamad@fedora:~/work/arch-pc/lab09 — gdb lab9-2
~/work/arch-pc/lab09
nassermohamad@fedora:~/work/arch-pc/lab09$ nasm -f elf -g -l lab9-2.lst lab9-2.asm
nassermohamad@fedora:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab9-2 lab9-2.o
nassermohamad@fedora:~/work/arch-pc/lab09$ gdb lab9-2
GNU gdb (Fedora Linux) 16.2-3.fc42
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab9-2...
(gdb) r
Starting program: /home/nassermohamad/work/arch-pc/lab09/lab9-2

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.fedoraproject.org/>
Enable debuginfod for this session? (y or [n])
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
Hello, world!
[Inferior 1 (process 9179) exited normally]
(gdb)
```

Рисунок 2.6: Запуск программы lab9-2.asm в отладчике

Для более подробного анализа программы, установил точку остановки на метке „start“, с которой начинается выполнение любой ассемблерной программы, и запустил ее. Затем просмотрел дизассемблированный код программы.(рис. 2.7) (рис. 2.8)

```
nassermohamad@fedora:~/work/arch-pc/lab09 — gdb lab9-2
~/work/arch-pc/lab09

<https://debuginfod.fedoraproject.org/>
Enable debuginfod for this session? (y or [n])
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
Hello, world!
[Inferior 1 (process 9179) exited normally]
(gdb) break _start
Breakpoint 1 at 0x08048080: file lab9-2.asm, line 12.
(gdb) r
Starting program: /home/nassermohamad/work/arch-pc/lab09/lab9-2

Breakpoint 1, _start () at lab9-2.asm:12
12      mov eax, 4
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08048080 <+0>:      mov     $0x4,%eax
0x08048085 <+5>:      mov     $0x1,%ebx
0x0804808a <+10>:     mov     $0x8049000,%ecx
0x0804808f <+15>:     mov     $0x8,%edx
0x08048094 <+20>:     int     $0x80
0x08048096 <+22>:     mov     $0x4,%eax
0x0804809b <+27>:     mov     $0x1,%ebx
0x080480a0 <+32>:     mov     $0x8049008,%ecx
0x080480a5 <+37>:     mov     $0x7,%edx
0x080480aa <+42>:     int     $0x80
0x080480ac <+44>:     mov     $0x1,%eax
0x080480b1 <+49>:     mov     $0x0,%ebx
0x080480b6 <+54>:     int     $0x80
End of assembler dump.
(gdb) █
```

Рисунок 2.7: Дизассемблированный код

```
nassermohamad@fedora:~/work/arch-pc/lab09 — gdb lab9-2
~/work/arch-pc/lab09

0x0804808a <+10>:  mov    $0x8049000,%ecx
0x0804808f <+15>:  mov    $0x8,%edx
0x08048094 <+20>:  int    $0x80
0x08048096 <+22>:  mov    $0x4,%eax
0x0804809b <+27>:  mov    $0x1,%ebx
0x080480a0 <+32>:  mov    $0x8049008,%ecx
0x080480a5 <+37>:  mov    $0x7,%edx
0x080480aa <+42>:  int    $0x80
0x080480ac <+44>:  mov    $0x1,%eax
0x080480b1 <+49>:  mov    $0x0,%ebx
0x080480b6 <+54>:  int    $0x80
End of assembler dump.
(gdb) set disassembly-flavor intel
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08048080 <+0>:  mov    eax,0x4
    0x08048085 <+5>:  mov    ebx,0x1
    0x0804808a <+10>:  mov    ecx,0x8049000
    0x0804808f <+15>:  mov    edx,0x8
    0x08048094 <+20>:  int    0x80
    0x08048096 <+22>:  mov    eax,0x4
    0x0804809b <+27>:  mov    ebx,0x1
    0x080480a0 <+32>:  mov    ecx,0x8049008
    0x080480a5 <+37>:  mov    edx,0x7
    0x080480aa <+42>:  int    0x80
    0x080480ac <+44>:  mov    eax,0x1
    0x080480b1 <+49>:  mov    ebx,0x0
    0x080480b6 <+54>:  int    0x80
End of assembler dump.
(gdb) █
```

Рисунок 2.8: Дизассемблированный код в режиме интел

Для проверки точки останова по имени метки '_start', использовал команду „info breakpoints“ (сокращенно „i b“). Затем установил еще одну точку останова по адресу инструкции, определив адрес предпоследней инструкции „mov ebx, 0x0“. (рис. 2.9)

```
nassermohamad@fedora:~/work/arch-pc/lab09 — gdb lab9-2
~/work/arch-pc/lab09

Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffce50 0xffffce50
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0

B+>0x8048080 <_start> mov eax,0x4
0x8048085 <_start+5> mov ebx,0x1
0x804808a <_start+10> mov ecx,0x8049000
0x804808f <_start+15> mov edx,0x8
0x8048094 <_start+20> int 0x80
0x8048096 <_start+22> mov eax,0x4
0x804809b <_start+27> mov ebx,0x1
0x80480a0 <_start+32> mov ecx,0x8049008

native process 9183 (asm) In: _start L12 PC: 0x8048080
(gdb) layout regs
(gdb) b *0x8049031
Breakpoint 2 at 0x8049031
(gdb) i b
Num Type Disp Enb Address What
1 breakpoint keep y 0x08048080 lab9-2.asm:12
breakpoint already hit 1 time
2 breakpoint keep y 0x08049031
(gdb) 
```

Рисунок 2.9: Точка остановки

В отладчике GDB можно просматривать содержимое ячеек памяти и регистров, а также изменять значения регистров и переменных. Выполнил 5 инструкций с помощью команды „stepi“ (сокращенно „si“) и отследил изменение значений регистров. (рис. 2.10) (рис. 2.11)

```
nassermohamad@fedora:~/work/arch-pc/lab09 — gdb lab9-2
~/work/arch-pc/lab09

Register group: general
eax      0x4      4
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffce50 0xffffce50
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0

B+ 0x8048080 <_start>   mov     eax,0x4
>0x8048085 <_start+5>   mov     ebx,0x1
0x804808a <_start+10>   mov     ecx,0x8049000
0x804808f <_start+15>   mov     edx,0x8
0x8048094 <_start+20>   int     0x80
0x8048096 <_start+22>   mov     eax,0x4
0x804809b <_start+27>   mov     ebx,0x1
0x80480a0 <_start+32>   mov     ecx,0x8049008

native process 9183 (asm) In: _start L13 PC: 0x8048085
--Type <RET> for more, q to quit, c to continue without paging--
eflags    0x202      [ IF ]
cs         0x23      35
ss         0x2b      43
ds         0x2b      43
es         0x2b      43
fs         0x0       0
gs         0x0       0
(gdb) si
(gdb) 
```

Рисунок 2.10: Изменение регистров

```
nassermohamad@fedora:~/work/arch-pc/lab09 — gdb lab9-2
~/work/arch-pc/lab09

Register group: general
eax      0x8      8
ecx      0x8049000 134516736
edx      0x8      8
ebx      0x1      1
esp      0xffffce50 0xffffce50
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0

0+ 0x8048080 <_start>    mov     eax,0x4
0x8048085 <_start+5>    mov     ebx,0x1
0x804808a <_start+10>   mov     ecx,0x8049000
0x804808f <_start+15>   mov     edx,0x8
0x8048094 <_start+20>   int     0x80
>0x8048096 <_start+22>   mov     eax,0x4
0x804809b <_start+27>   mov     ebx,0x1
0x80480a0 <_start+32>   mov     ecx,0x8049008

native process 9183 (asm) In: _start L17 PC: 0x8048096
ds      0x2b      43
es      0x2b      43
fs      0x0      0
gs      0x0      0
(gdb) si
(gdb) si
(gdb) si
(gdb) si
(gdb) siHello,
(gdb) █
```

Рисунок 2.11: Изменение регистров

Просмотрел значение переменной msg1 по имени и получил нужные данные.

Просмотрел значение переменной msg1 по имени и получил нужные данные.

Для изменения значения регистра или ячейки памяти использовал команду set, указав имя регистра или адрес в качестве аргумента. Изменил первый символ переменной msg1. (рис. 2.12)


```
nassermohamad@fedora:~/work/arch-pc/lab09 — gdb lab9-2
~/work/arch-pc/lab09

Register group: general
eax      0x8      8
ecx      0x8049000 134516736
edx      0x8      8
ebx      0x1      1
esp      0xffffce50 0xffffce50
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0

B+ 0x8048080 <_start>    mov     eax,0x4
0x8048085 <_start+5>    mov     ebx,0x1
0x804808a <_start+10>   mov     ecx,0x8049000
0x804808f <_start+15>   mov     edx,0x8
0x8048094 <_start+20>   int     0x80
>0x8048096 <_start+22>   mov     eax,0x4
0x804809b <_start+27>   mov     ebx,0x1
0x80480a0 <_start+32>   mov     ecx,0x8049008

native process 9183 (asm) In: _start L17 PC: 0x8048096
(gdb) x/1sb 0x804a008
0x804a008: <error: Cannot access memory at address 0x804a008>
(gdb) set {char}&msg1='h'
(gdb) x/1sb &msg1
0x8049000 <msg1>: "hello, "
(gdb) set {char}0x804a008='L'
Cannot access memory at address 0x804a008
(gdb) x/1sb 0x804a008
0x804a008: <error: Cannot access memory at address 0x804a008>
(gdb) 
```

Рисунок 2.12: Изменение значения переменной

Для изменения значения регистра или ячейки памяти использовал команду `set`, указав имя регистра или адрес в качестве аргумента. Изменил первый символ переменной `msg1`. (рис. 2.13)

```
nassermohamad@fedora:~/work/arch-pc/lab09 — gdb lab9-2
~/work/arch-pc/lab09

Register group: general
eax      0x8      8
ecx      0x8049000 134516736
edx      0x8      8
ebx      0x1      1
esp      0xffffce50 0xffffce50
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0

B+ 0x8048080 <_start>    mov     eax,0x4
    0x8048085 <_start+5>  mov     ebx,0x1
    0x804808a <_start+10> mov     ecx,0x8049000
    0x804808f <_start+15> mov     edx,0x8
    0x8048094 <_start+20> int      0x80
>0x8048096 <_start+22>  mov     eax,0x4
    0x804809b <_start+27> mov     ebx,0x1
    0x80480a0 <_start+32> mov     ecx,0x8049000

native process 9183 (asm) In: _start L17 PC: 0x8048096
$3 = 134516736
(gdb) p/x $ecx
$4 = 0x8049000
(gdb) p/s $edx
$5 = 8
(gdb) p/t $edx
$6 = 1000
(gdb) p/x $edx
$7 = 0x8
(gdb) 
```

Рисунок 2.13: Вывод значения регистра

С помощью команды set изменил значение регистра ebx на нужное значение. (рис. 2.14)

The screenshot shows a GDB terminal window with the title bar "nassermohamad@fedora:~/work/arch-pc/lab09 — gdb lab9-2". The window is divided into two main sections. The top section, titled "Register group: general", displays the values of several registers:

Register	Value	Comment
eax	0x8	8
ecx	0x8049000	134516736
edx	0x8	8
ebx	0x2	2
esp	0xffffce50	0xffffce50
ebp	0x0	0x0
esi	0x0	0
edi	0x0	0

. The bottom section shows assembly code with addresses and instructions:

```
B+ 0x8048080 <_start>    mov     eax,0x4
0x8048085 <_start+5>    mov     ebx,0x1
0x804808a <_start+10>   mov     ecx,0x8049000
0x804808f <_start+15>   mov     edx,0x8
0x8048094 <_start+20>   int     0x80
>0x8048096 <_start+22>   mov     eax,0x4
0x804809b <_start+27>   mov     ebx,0x1
0x80480a0 <_start+32>   mov     ecx,0x8049008
```

. The bottom status bar indicates "native process 9183 (asm) In: _start" and "L17 PC: 0x8048096". The command history at the bottom shows:

```
$6 = 1000
(gdb) p/x $edx
$7 = 0x8
(gdb) set $ebx='2'
(gdb) p/s $ebx
$8 = 50
(gdb) set $ebx=2
(gdb) p/s $ebx
$9 = 2
(gdb)
```

Рисунок 2.14: Вывод значения регистра

Скопировал файл lab8-2.asm, созданный во время выполнения лабораторной работы №8, который содержит программу для вывода аргументов командной строки. Создал исполняемый файл из скопированного файла.

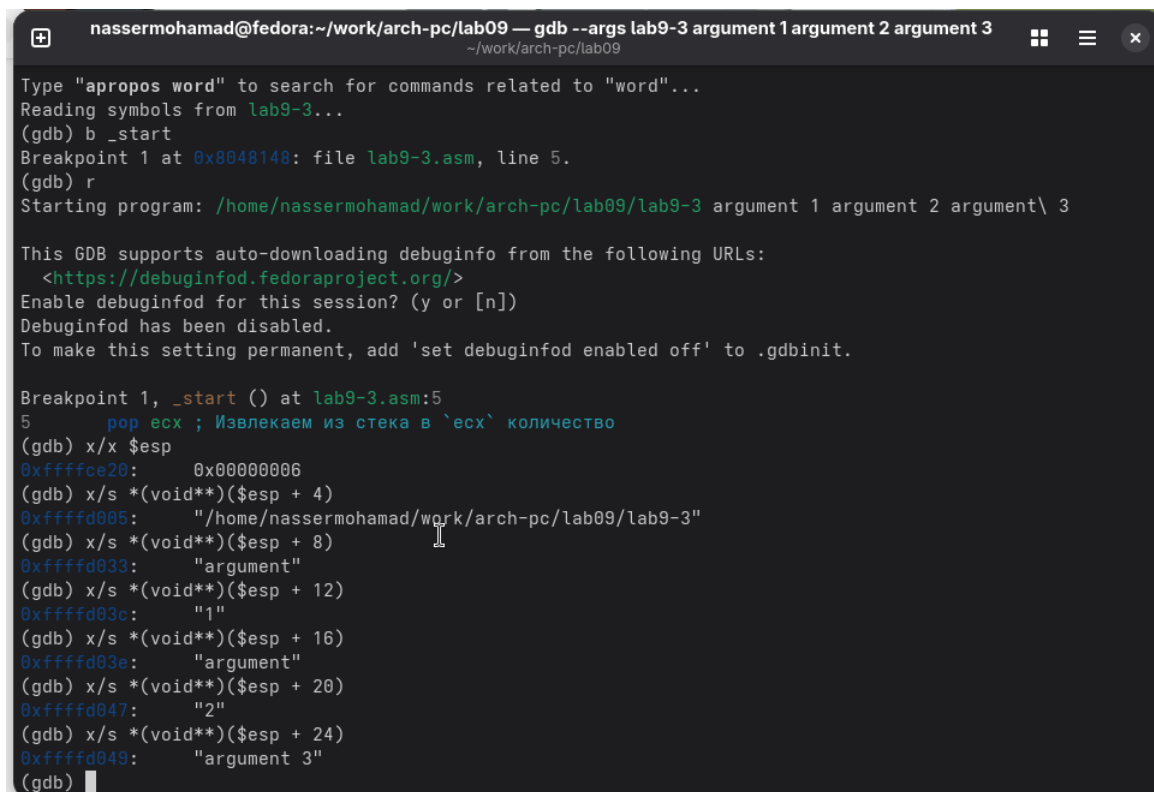
Для загрузки программы с аргументами в gdb использовал ключ `-args` и загрузил исполняемый файл в отладчик с указанными аргументами.

Установил точку останова перед первой инструкцией программы и запустил ее.

Адрес вершины стека, содержащий количество аргументов командной строки (включая имя программы), хранится в регистре `esp`. По этому адресу находится число, указывающее количество аргументов. В данном случае видно, что количество аргументов равно 5, включая имя программы lab9-3 и сами аргументы: аргумент1, аргумент2 и „аргумент 3“.

Просмотрел остальные позиции стека. По адресу `[esp+4]` находится адрес в

памяти, где располагается имя программы. По адресу [esp+8] хранится адрес первого аргумента, по адресу [esp+12] - второго и так далее. (рис. 2.15)



```
nassermohamad@fedora:~/work/arch-pc/lab09 — gdb --args lab9-3 argument 1 argument 2 argument 3
~/work/arch-pc/lab09

Type "apropos word" to search for commands related to "word"...
Reading symbols from lab9-3...
(gdb) b _start
Breakpoint 1 at 0x8048148: file lab9-3.asm, line 5.
(gdb) r
Starting program: /home/nassermohamad/work/arch-pc/lab09/lab9-3 argument 1 argument 2 argument\ 3

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.fedoraproject.org/>
Enable debuginfod for this session? (y or [n])
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.

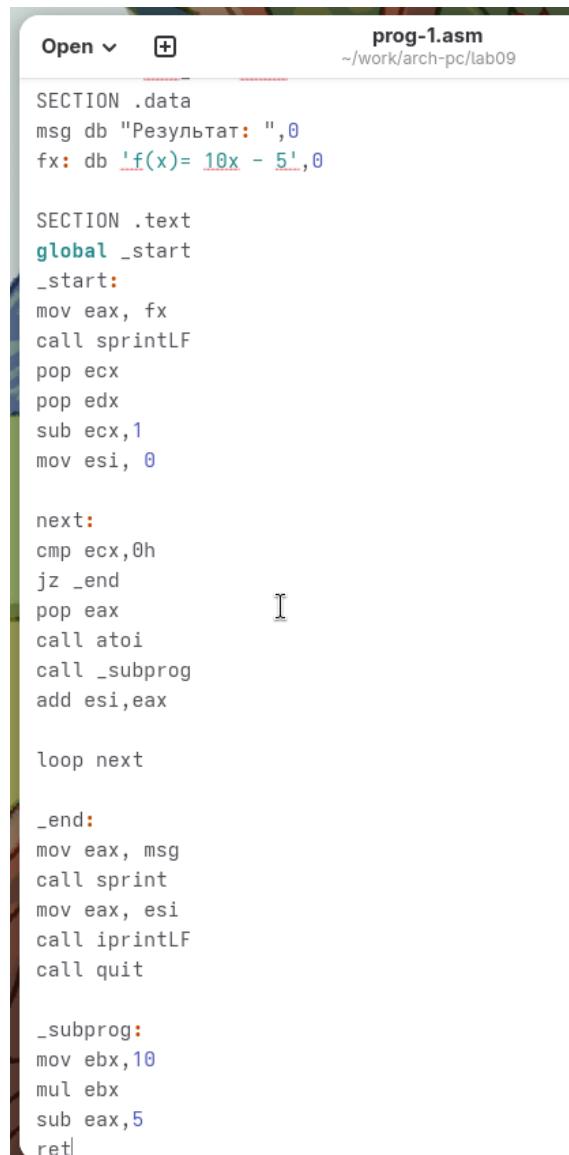
Breakpoint 1, _start () at lab9-3.asm:5
5      pop ecx ; Извлекаем из стека в `ecx` количество
(gdb) x/x $esp
0xfffffce20: 0x00000006
(gdb) x/s *(void**)(esp + 4)
0xfffffd005: "/home/nassermohamad/work/arch-pc/lab09/lab9-3"
(gdb) x/s *(void**)(esp + 8)
0xfffffd033: "argument"
(gdb) x/s *(void**)(esp + 12)
0xfffffd03c: "1"
(gdb) x/s *(void**)(esp + 16)
0xfffffd03e: "argument"
(gdb) x/s *(void**)(esp + 20)
0xfffffd047: "2"
(gdb) x/s *(void**)(esp + 24)
0xfffffd049: "argument 3"
(gdb)
```

Рисунок 2.15: Вывод значения регистра

Шаг изменения адреса равен 4, так как каждый следующий адрес на стеке находится на расстоянии 4 байт от предыдущего ([esp+4], [esp+8], [esp+12]).

2.1 Самостоятельное задание

Преобразовал программу из лабораторной работы №8 (Задание №1 для самостоятельной работы), реализовав вычисление значения функции $f(x)$ как подпрограмму. (рис. 2.16) (рис. 2.17)



```

Open v + prog-1.asm
~/work/arch-pc/lab09

SECTION .data
msg db "Результат: ",0
fx: db 'f(x)= 10x - 5',0

SECTION .text
global _start
_start:
mov eax, fx
call sprintf
pop ecx
pop edx
sub ecx,1
mov esi, 0

next:
cmp ecx,0h
jz _end
pop eax
call atoi
call _subprog
add esi,eax

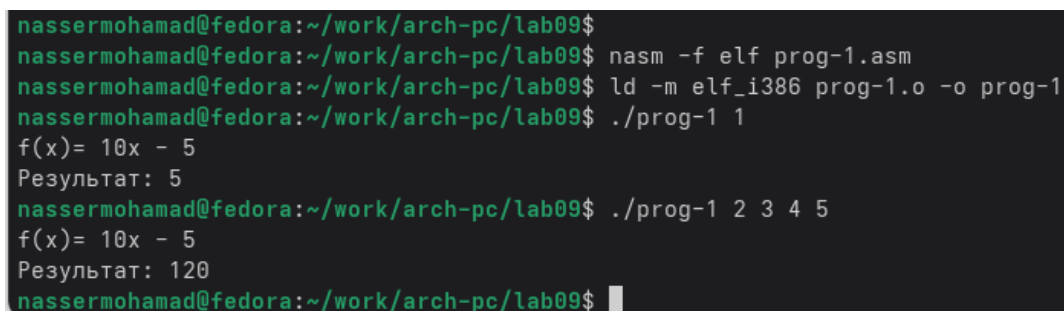
loop next

_end:
mov eax, msg
call sprintf
mov eax, esi
call iprintLF
call quit

_subprog:
mov ebx,10
mul ebx
sub eax,5
ret

```

Рисунок 2.16: Программа в файле prog-1.asm



```

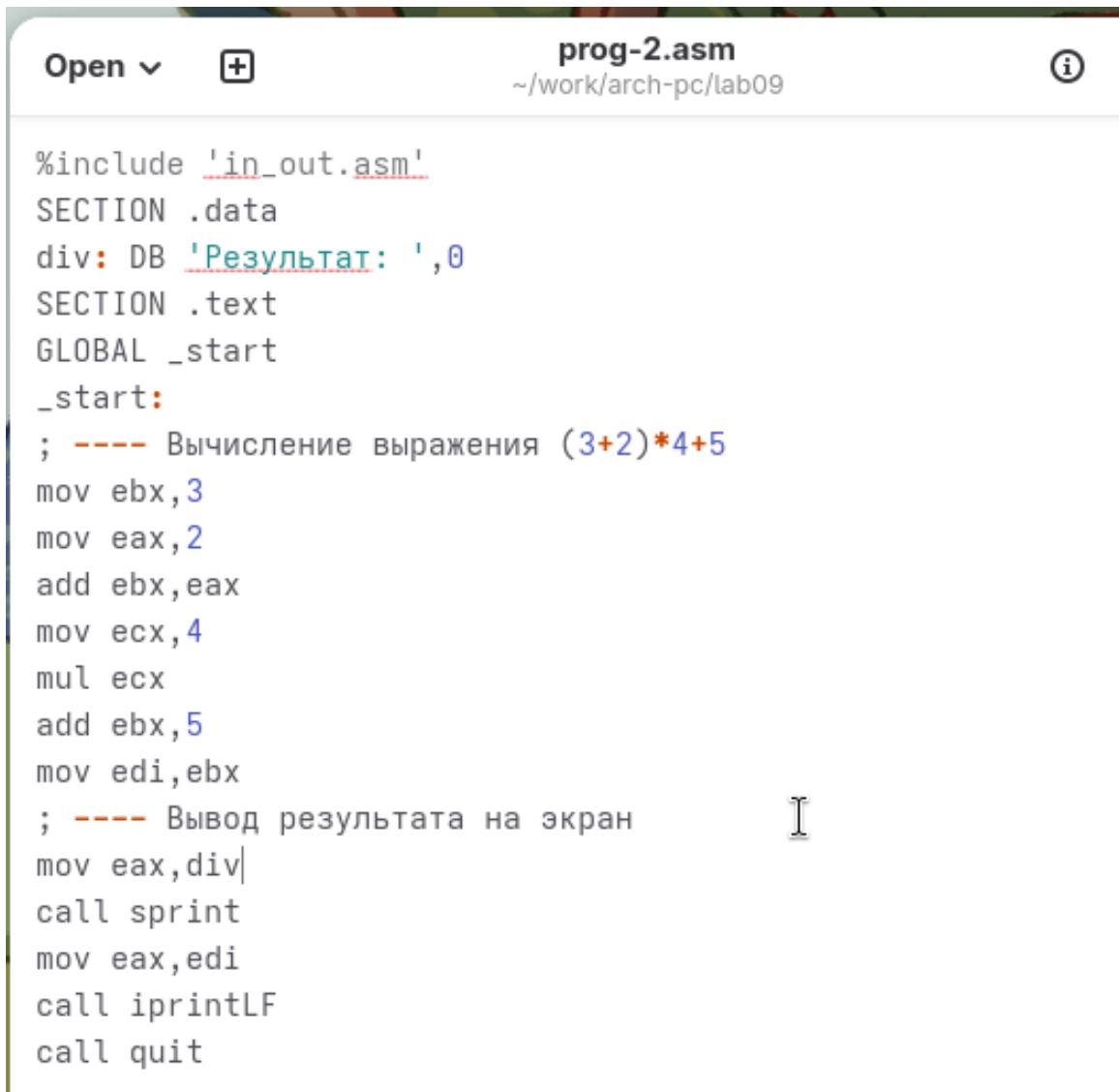
nassermohamad@fedora:~/work/arch-pc/lab09$
nassermohamad@fedora:~/work/arch-pc/lab09$ nasm -f elf prog-1.asm
nassermohamad@fedora:~/work/arch-pc/lab09$ ld -m elf_i386 prog-1.o -o prog-1
nassermohamad@fedora:~/work/arch-pc/lab09$ ./prog-1 1
f(x)= 10x - 5
Результат: 5
nassermohamad@fedora:~/work/arch-pc/lab09$ ./prog-1 2 3 4 5
f(x)= 10x - 5
Результат: 120
nassermohamad@fedora:~/work/arch-pc/lab09$

```

Рисунок 2.17: Запуск программы prog-1.asm

В листинге приведена программа вычисления выражения $(3 + 2) * 4 + 5$. При запуске данная программа дает неверный результат. Проверил это, анализируя изменения значений регистров с помощью отладчика GDB.

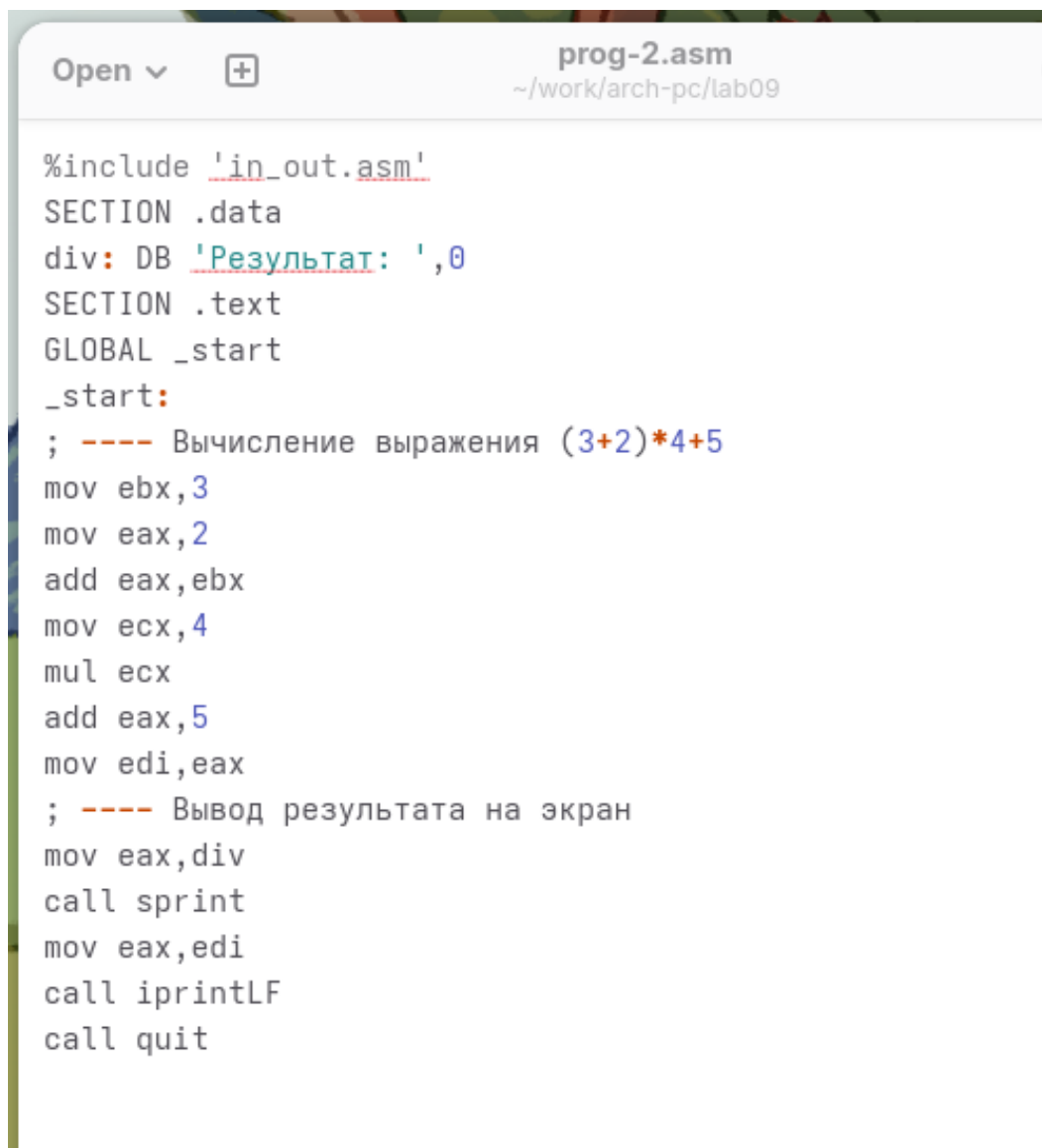
Определил ошибку - перепутан порядок аргументов у инструкции `add`. Также обнаружил, что по окончании работы в `edi` отправляется `ebx` вместо `eax`. (рис. 2.18)



```
Open v [icon] prog-2.asm [info]
~/work/arch-pc/lab09

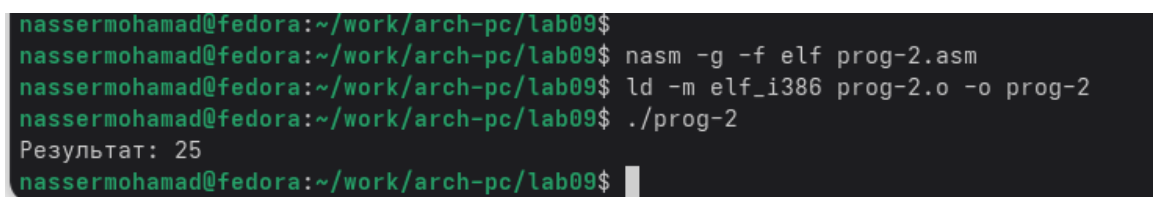
%include 'in_out.asm'
SECTION .data
div: DB 'Результат: ',0
SECTION .text
GLOBAL _start
_start:
; ---- Вычисление выражения (3+2)*4+5
mov ebx,3
mov eax,2
add ebx,eax
mov ecx,4
mul ecx
add ebx,5
mov edi,ebx
; ---- Вывод результата на экран
mov eax,div
call sprint
mov eax,edi
call iprintLF
call quit
```

Рисунок 2.18: Код с ошибкой



```
Open ▾  prog-2.asm  
~/work/arch-pc/lab09  
  
%include 'in_out.asm'  
SECTION .data  
div: DB 'Результат: ',0  
SECTION .text  
GLOBAL _start  
_start:  
; ---- Вычисление выражения (3+2)*4+5  
mov ebx,3  
mov eax,2  
add eax,ebx  
mov ecx,4  
mul ecx  
add eax,5  
mov edi,eax  
; ---- Вывод результата на экран  
mov eax,div  
call sprint  
mov eax,edi  
call iprintLF  
call quit
```

Рисунок 2.20: Код исправлен



```
nassermohamad@fedora:~/work/arch-pc/lab09$  
nassermohamad@fedora:~/work/arch-pc/lab09$ nasm -g -f elf prog-2.asm  
nassermohamad@fedora:~/work/arch-pc/lab09$ ld -m elf_i386 prog-2.o -o prog-2  
nassermohamad@fedora:~/work/arch-pc/lab09$ ./prog-2  
Результат: 25  
nassermohamad@fedora:~/work/arch-pc/lab09$
```

Рисунок 2.21: Проверка работы

3 Выводы

Освоили работу с подпрограммами и отладчиком.